EXECUTIVE BRIEFING · 15 MINUTES

Context Is a Budget

Reducing Token Usage in AI-Assisted Development

Patterns that turn an expensive, unpredictable habit into a governable, measurable engineering practice.

THE OPPORTUNITY

Coding assistants are now infrastructure

55%
OF DEV TIME

Median % of coding time augmented by AI in mature engineering orgs (2025 surveys, internal benchmarks).

2–4×
THROUGHPUT UPLIFT

On well-scoped tasks (refactors, tests, glue code) — the upper end requires *workflow*, not just licenses.

$10–50k
PER 50 DEVS / MONTH

Realistic burn rate on agentic tooling without governance. Easily 3× higher in heavy agent-mode shops.

The question is no longer *whether* to deploy AI assistants. It is **how to operate them at scale without losing the upside to waste, risk, and inconsistency.**

WHY THIS MATTERS NOW

Three silent taxes on every AI workflow

💰 Cost drift

Premium-request quotas burn unevenly across teams. Top 10% of users typically consume 60–70% of spend.

Without governance: month-to-month variance >40%.

⏱️ Latency creep

Bigger contexts = slower responses. A 30-turn chat is 3–5× slower than a fresh one.

Felt as: developer frustration, abandoned sessions, fewer iterations.

📉 Context rot

Recall and reasoning **degrade** as the context window fills — even within the model's advertised limit. Quality goes *down*.

Source: Anthropic & Chroma research, 2024–25.

> All three taxes have the same root cause: **treating context as free real estate** instead of a finite budget.

WHERE THE BUDGET GOES

Anatomy of an AI request

BUCKET	TYPICAL SHARE	WHY IT BLOATS	LEVER
System prompt / instructions	5–15%	Copy-pasted boilerplate	Custom instructions, skills
Tool / function schemas	10–40%	Re-sent **every** turn; unused MCP servers	Tool hygiene
Retrieved files & code chunks	20–60%	Whole-file dumps; broad retrieval	Context engineering
Conversation history	10–30%	Grows linearly without compaction	Auto-compact
Model output	5–20%	"Let me explain what I'm about to do…"	Output discipline

Most teams are surprised which bucket dominates theirs. **Profile first, optimize second.**

THE PLAYBOOK

Eight levers that move the needle

A · Context engineering

Scope every ask. **#file:** over broad retrieval.

B · Prompt caching

Static at top, dynamic at bottom. ~10% of base cost on cache hits.

C · Tool / MCP hygiene

Disable unused servers. Compact tool responses by default.

D · Instructions & skills

Codify conventions once. Path-scoped instructions for stack-specific rules.

E · Model routing

Cheap for routine, mid for real work, top-tier only when stuck.

F · Output discipline

Diffs, not whole files. No preamble. Structured output.

G · Repo hygiene

.gitignore the indexer. Summary comments at module tops.

H · Observability

Latency as a token meter. CI lints for prompt/instruction bloat.

None are novel. None are hard. The **compounding** across all eight is what moves a $30k/mo bill to $12k/mo with *better* output quality.

THE MULTIPLIERS

Three workflow patterns that compound on top

1 · The Ralph loop

Cheap model + small fresh context + on-disk state (`TODO.md`).

- One iteration per task
- State on disk, not in chat
- Restartable, auditable in `git log`

Use for: chore lists, repetitive migrations, test backfill.

2 · Auto-compact

Summarize a long chat to `plan.md`; continue in a new chat from the summary.

- 4 KB summary keeps ~95% of the signal
- ~3% of the cost
- Summary becomes a cacheable prefix

Use for: any session past ~60% context.

3 · Agent handover

Planner → Implementer → Reviewer. Three short chats, three model choices, one shared artifact.

- ~5–8 premium reqs per feature
- vs. 30+ on a single mega-chat
- Auditable paper trail

Use for: any non-trivial feature.

These are **composable**. Mature teams run all three on different workloads in the same week.

PATTERN DEEP-DIVE

Planner → Implementer → Reviewer

Planner

Top-tier model · 1 call

Produces plan.md: goal, acceptance criteria, tasks, out-of-scope, risks. **No code.**

→

Implementer

Cheap model · agent mode

Ralph loop on plan.md: pick task, edit, test, check box, commit. Sees only plan.md + touched files.

→

Reviewer

Top-tier model · 1 call

Sees plan.md + diff only. Marks each criterion PASS/FAIL. VERDICT: APPROVE / REQUEST CHANGES.

The discipline

Only the **handover artifact** (plan.md, the diff, the review notes) crosses the boundary. **Never chat history.**

Each agent stays small, focused, cheap.

The economics

~5–8 premium requests for an end-to-end feature *with a paper trail*.

Compare: a single Opus mega-chat — 30+ requests at 10× the multiplier.

ENTERPRISE CONCERNS

Governance, security, and the audit trail

What we control at the repo

- **Custom instructions** — committed, reviewed, versioned. Same artifact as code.
- **Path-scoped rules** — load only when relevant; no ambient cost.
- **Content exclusions** — secrets, fixtures, generated code never reach the model.
- **.gitignore + indexer policies** — only the right surface is retrievable.

What we observe at the platform

- **Per-user, per-team usage** — Copilot Metrics API, vendor dashboards.
- **Per-turn telemetry** — model, latency, status (Output panel `ccreq` lines).
- **CI lints** — fail builds when instruction/prompt files exceed budget.
- **Audit trail** — every implementer commit is one task = one revertible change.

> "The worst tokens are the ones you're paying for and not noticing."
>
> — a slogan that sounds glib until you see the first month's bill.

ORDER-OF-MAGNITUDE CHECK

What does discipline buy you?

Team size (developers)

50

Complex tasks / dev / week (features, refactors, multi-file)

3

Simple tasks / dev / week (small edits, tests, glue code)

15

Default workflow today

Mixed models, no routing discipline

After applying the playbook

Routing + caching + handover (full)

Model turns / month — today	**33,000 turns**
Model turns / month — after	**10,800 turns**
Context tokens / month — today	**1.20 B**
Context tokens / month — after	**108.0 M**
Reduction in context consumed	**91% less context**
Engineer wait-time reclaimed	**185 hrs / mo**

Routing + caching + handover removes **22,200 model turns/month** (91% less context shipped to the LLM) — recovering roughly **185 engineer-hours** of waiting and re-prompting. The compounding win is fewer *turns to outcome*, not just cheaper turns.

Numbers are illustrative — built to give an order-of-magnitude sense, not a quote. Real engagements include a measurement window first.

IF WE HAVE TIME / IN A FOLLOW-UP

Live demos available on request

🎯 Context engineering — the 10× delta

Same refactor, two prompts. Vague agent-mode ask vs. a `#file:`-pinned ask. Compare diffs and request payload sizes side by side.

Time: 3 min · **Shows:** immediate cost & quality win.

🔁 Ralph loop on a real backlog

A 5-item TODO.md drained autonomously by a cheap model. Per-task commits, all tests green, fully reset-able.

Time: 5 min · **Shows:** directed AI workload, audit trail.

📋 Planner → Implementer → Reviewer

End-to-end feature delivered across three short chats. Count the premium requests as it runs.

Time: 6 min · **Shows:** paper trail + cost discipline together.

🚦 Observability & CI lints

The 30-line lint script that fails CI on instruction-file bloat. Demonstrate it passing, then bloat a file and watch it fail.

Time: 3 min · **Shows:** guardrails that survive PR reviews.

All four can run in **a single 60-min hands-on follow-up** using your own repo, with your security boundaries respected.

HOW A ROLLOUT ACTUALLY LOOKS

Phase 1 → Phase 2 → Phase 3

PHASE 1

Measure & standardize

- Baseline usage from vendor dashboards + Metrics API
- Identify top 3 repos by AI activity
- Land **AGENTS.md / copilot-instructions.md** in each
- Add path-scoped instructions for the dominant stack
- Disable unused MCP servers org-wide

Outcome: visibility + a defensible default config.

PHASE 2

Adopt the patterns

- Pilot the **Ralph loop** on one team's chore backlog
- Pilot the **handover pattern** on one feature per squad
- Train champions; capture before/after telemetry
- Land the **CI lint scripts** in those repos

Outcome: two teams with measured uplift & reduced spend.

PHASE 3

Scale & govern

- Roll patterns out across the org's top-20 repos
- Add lint scripts to org-wide CI templates
- Establish a **model routing policy** (default mid-tier; top-tier requires reason)
- Recurring cost & latency review on a shared dashboard

Outcome: repeatable, governed practice — not a tribal habit.

NEXT STEPS

From expensive habit to disciplined practice

A two-week assessment on one of your repos:
baseline measurement → playbook fit → quantified projection.

Week 1

Baseline current usage + collect `ccreq` telemetry from 1–2 pilot teams.

Week 2

Apply 3 levers + 1 workflow pattern. Measure delta on the same workloads.

Out of it

A short report: real numbers for **your** code, not industry averages, and a phased rollout plan.

> Treat your token budget the way you treat your cloud bill.

The discipline already exists in your org. We just port it over.